

ML + Regression

1. Types of Machine Learning

ML = teaching a computer from data instead of writing rules manually. There are 4 main types:

Supervised Learning → labelled data

You give the model data where you already know the answer. It learns the pattern, then predicts on new data. **Example:** Predicting house price from area + bedrooms. You give 10,000 past sales (area → price). Model learns the formula, predicts price for a new house.

Semi-Supervised → little labelled, lots unlabelled

Label only ~5–10% of data, leave the rest blank. Model uses the labelled part as anchor, figures out the rest. (*think: learning from a few solved examples*)

Example: Google Photos. Labelling every photo by hand is impossible. So they label 1M photos, leave 50B unlabelled. Works surprisingly well.

Unsupervised Learning → no labels at all

Model just looks at raw data and tries to find hidden structure / groupings.

Example: Amazon customer segmentation. Nobody told the model "group 1 = students". It found that on its own — customers who buy similar stuff get grouped together.

- Main technique = Clustering (K-Means, DBSCAN)
- Also: Dimensionality reduction (PCA)

Reinforcement Learning → learns by trial & error

Agent makes decisions, gets reward for good moves, penalty for bad. Over millions of tries it figures out the best strategy. (*think: training a dog with treats*)

Example: AlphaGo. It played Go against itself millions of times. Win = +reward, Lose = penalty. Eventually beat world champions.

Total Reward = $R_1 + \gamma R_2 + \gamma^2 R_3 + \dots$ (γ = discount factor, how much future matters)

2. Regression — Predicting Numbers

Regression = supervised learning where the output is a continuous number (not a category).

Linear Regression → output is a number (salary, price, temperature)

Logistic Regression → output is a class (spam/not spam, fraud/not fraud) ← confusing name but it's classification!

Linear Regression formula

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

$\hat{y}$ = prediction, β_0 = baseline (intercept), β_1 = slope / weight for feature x_1

Quick example: Score from study hours

Data you collected:

Hours studied:	1	2	3	4	5	6	7
----------------	---	---	---	---	---	---	---

Exam score:	35	45	50	60	65	72	80
-------------	----	----	----	----	----	----	----

After fitting: $\hat{y} = 28 + 7.5 \times \text{hours}$

→ $\beta_0 = 28$ means: even with 0 hours you'd score ~28 (baseline)

→ $\beta_1 = 7.5$ means: each extra hour adds ~7.5 marks

Predict for 5 hours: $\hat{y} = 28 + 7.5 \times 5 = 65.5$ marks ✓

How does the model 'learn' the β values?

Two ways:

- OLS (Ordinary Least Squares) — solves it directly with math. Fast, works for small data.
- Gradient Descent — iteratively adjusts β to reduce error. Used for big datasets.

$$\beta := \beta - \alpha \times (\partial \text{Loss} / \partial \beta) \quad \alpha = \text{learning rate (step size)}$$

Gradient Descent — How the Model Actually Learns

Gradient descent is the algorithm that minimises the cost function. Instead of guessing β values randomly, it starts somewhere and systematically walks toward the lowest point of the cost curve.

The Core Idea — Walking Downhill

Gradient descent is the algorithm that actually minimizes the cost function. It works like walking downhill. We start with random values of β_0 and β_1 , calculate the cost, then take small steps in the direction that reduces the cost the most. We keep doing this until we reach the bottom (minimum cost). The size of each step is controlled by the learning rate.

The Formula

$$\beta_1 := \beta_1 - \alpha \times (\partial \text{MSE} / \partial \beta_1)$$

Step-by-Step Walkthrough

Data: hours studied vs exam score

True best values (OLS answer): $\beta_0 = 28$, $\beta_1 = 7.5$

Step 0: Start random $\beta_1 = 12.0$ MSE = 312.4 ← way off

Step 1: Gradient = +18.6 $\beta_1 = 12.0 - 0.1 \times 18.6 = 10.1$ MSE = 198.3

Step 2: Gradient = +11.2 $\beta_1 = 10.1 - 0.1 \times 11.2 = 9.0$ MSE = 104.7

Step 3: Gradient = +5.1 $\beta_1 = 9.0 - 0.1 \times 5.1 = 8.5$ MSE = 44.2

Step 4: Gradient = +1.9 $\beta_1 = 8.5 - 0.1 \times 1.9 = 8.3$ MSE = 18.1

Step 5: Gradient = +0.6 $\beta_1 = 8.3 - 0.1 \times 0.6 = 8.2$ MSE = 9.4

...

Step 40: Gradient ≈ 0.0 $\beta_1 \approx 7.5$ MSE ≈ 6.1 ← converged ✓

Every step, MSE drops. The gradient gets smaller as we approach the minimum, so the steps naturally get smaller too. Eventually it basically stops moving — that's convergence.

Learning Rate α — The Most Important Setting

The learning rate controls how big each step is. Getting this right is crucial.

α too LARGE (e.g. $\alpha = 2.0$):

Step 1: $\beta_1 = 12.0 \rightarrow 12.0 - 2 \times 18.6 = -25.2$ (jumped past minimum!)

Step 2: $\beta_1 = -25.2 \rightarrow$ overshoots other way $\rightarrow$ keeps bouncing, never settles

α too SMALL (e.g. $\alpha = 0.0001$):

Takes thousands of tiny steps to get there. Wastes time.

Fine mathematically, just slow.

α just right (e.g. $\alpha = 0.01$ to 0.1):

Steadily walks downhill, converges in 30-100 steps. ✓

Types of Gradient Descent

Batch Gradient Descent: uses ALL data points to compute the gradient in each step. Accurate but slow on large datasets.

Stochastic Gradient Descent (SGD): uses ONE random data point per step. Very fast, noisy path, but gets there. Used in deep learning.

Mini-Batch Gradient Descent: uses a small batch (e.g. 32 or 64 points) per step. Best of both — faster than batch, smoother than SGD. Most commonly used in practice.

3. Cost Function — How Wrong Is the Model?

The cost function measures how far off the model's predictions are. The model tries to minimize this.

MSE — Mean Squared Error (most common for regression)

$$\text{MSE} = (1/n) \times \sum (y_i - \hat{y}_i)^2$$

Actual scores:	50	65	80
Predicted scores:	52	60	83
Errors:	-2	5	-3
Squared errors:	4	25	9

$$\text{MSE} = (4 + 25 + 9) / 3 = 12.67$$

$\rightarrow$ Root of this (RMSE) $= \sqrt{12.67} \approx 3.56 \rightarrow$ model is off by ~3.5 marks on average

RSS & TSS $\rightarrow$ lead to R^2

$$\text{RSS} = \sum (y_i - \hat{y}_i)^2$$

$\leftarrow$ error AFTER fitting the model

$$\text{TSS} = \sum (y_i - \bar{y})^2$$

$\leftarrow$ total spread in the actual y values

$$R^2 = 1 - (\text{RSS} / \text{TSS})$$

explain?

$\leftarrow$ how much variance does model

```
y = [10, 20, 30],     $\bar{y} = 20$ ,     $\hat{y} = [12, 18, 33]$ 
```

```
RSS = (10-12)2 + (20-18)2 + (30-33)2 = 4 + 4 + 9 = 17
```

```
TSS = (10-20)2 + (20-20)2 + (30-20)2 = 100 + 0 + 100 = 200
```

```
 $R^2 = 1 - 17/200 = 0.915 \rightarrow$  model explains 91.5% of variation ✓
```

```
 $R^2 = 0 \rightarrow$  model is useless    |     $R^2 = 1 \rightarrow$  perfect fit
```

4. Bias–Variance Tradeoff + Regularization

The core problem

- High Bias (Underfitting) — model too simple, misses the pattern. Both train & test error are HIGH.
- High Variance (Overfitting) — model too complex, memorises training data. Train error LOW, test error HIGH.
- Goal: find the sweet spot where both are low.

Example: Predicting exam scores

```
Underfit:  $\hat{y} = 60$  (always predicts the mean, ignores hours studied)  
→ train error 15%, test error 15% ← both bad
```

```
Good fit:  $\hat{y} = 28 + 7.5 \times \text{hours}$   
→ train error 5%, test error 6% ← both good ✓
```

```
Overfit: 15-degree polynomial on 10 points  
→ train error 0.1%, test error 45% ← memorised training data
```

Fix for Overfitting → Regularization

Regularization adds a penalty for large β values to the cost function. Forces model to stay simple.

LASSO (L1) — can zero out useless features (automatic feature selection)

$$\text{Cost} = \text{MSE} + \lambda \times \sum |\beta_i|$$

Ridge (L2) — shrinks all coefficients but keeps them all

$$\text{Cost} = \text{MSE} + \lambda \times \sum \beta_i^2$$

```
Predicting salary from: [Experience, Education, Age, Hair_color, Shoe_size]
```

```
Without regularization:  $\beta = [2.1, 1.8, 0.5, 0.04, 0.02]$ 
```

```
After LASSO ( $\lambda=0.5$ ):  $\beta = [2.0, 1.9, 0.6, 0, 0]$   
→ Hair & Shoes zeroed out!
```

```
After Ridge ( $\lambda=0.5$ ):  $\beta = [1.85, 1.70, 0.45, 0.01, 0.008]$   
→ all kept but smaller
```

λ (lambda) controls strength. Higher λ = more penalty = simpler model. Choose λ via cross-validation.

5. VIF and Data Imbalance

VIF — Variance Inflation Factor

Checks if your features are too correlated with each other (**multicollinearity**). Correlated features make β coefficients unreliable.

$$\text{VIF}(x_j) = 1 / (1 - R_j^2) \quad R_j^2 = R^2 \text{ from regressing } x_j \text{ on all other features}$$

Rule of thumb: VIF < 5 = fine, VIF 5–10 = watch out, VIF > 10 = drop one of the correlated features

House price model features: Area, Bedrooms, Total_rooms, Age

Problem: Bedrooms is highly correlated with Total_rooms (obviously)

Regress Bedrooms on [Area, Total_rooms, Age] $\rightarrow R^2 = 0.92$

$\text{VIF}(\text{Bedrooms}) = 1 / (1 - 0.92) = 1 / 0.08 = 12.5 \leftarrow \text{too high!}$

Fix: Drop Total_rooms OR Bedrooms, keep only one.

Data Imbalance

When one class has way more samples than another. Classic example: fraud detection.

100,000 transactions: 99,500 genuine (99.5%) | 500 fraud (0.5%)

Dumb model: predict 'genuine' for everything

$\rightarrow$ Accuracy = 99.5% $\leftarrow$ looks great but catches ZERO frauds!

This is why accuracy alone is a terrible metric for imbalanced data.

Better metrics:

Precision = $TP / (TP + FP)$ $\rightarrow$ of all predicted positives, how many were actually positive?

Recall = $TP / (TP + FN)$ $\rightarrow$ of all actual positives, how many did we catch?

F1 Score = $2 \times P \times R / (P + R)$ $\rightarrow$ harmonic mean (use when both matter)

How to fix imbalance:

- SMOTE — generate synthetic minority class samples
- Undersampling — reduce majority class
- `class_weight='balanced'` in sklearn — penalises misclassifying minority more

1. fit the line

find m & c that minimise MSE on training data

2. check generalisation

compare train vs test error — are you in the sweet spot?

3. if overfitting

add regularization (Ridge or Lasso) — tune λ via cross-validation

4. if underfitting

add more features, increase complexity, or remove regularization